Внедрение ML в проект

otus.ru

REC Проверить, идет ли запись

Меня хорошо видно && слышно?

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в TG

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Цели вебинара

Что обсудим

1. Общая проблематика машинного обучения
2. Способы внедрения в Бэкенд
3. Основы ML

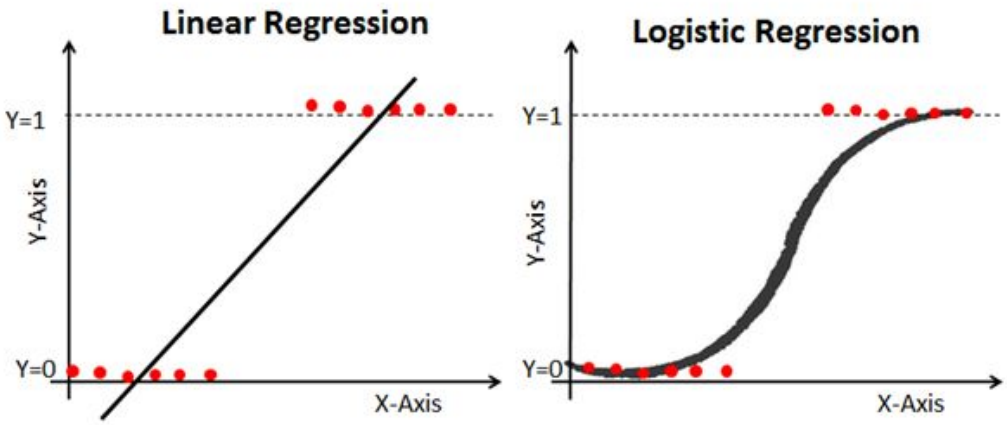
Меня зовут Александр Брут-Бруляко.

Я работаю DS и NLP инженером в Sber GigaLegal, там мы занимаемся обучением Гигачата Российскому праву и приложениями NLP к юридическому домену.
Закончил МехМат МГУ, преподавал в университете прикладную математику, к.э.н., доцент.

В DataScience, аналитике и IT данных я давно (больше 15 лет).

Общие проблемы ML

ML - мы строим модели, чтобы описывать реальность



модель - всегда упрощение реальности

Надо определить какой виды задачи мы решаем

Классификация

- назначаем объектам метку класса или вероятности принадлежности к классам

Регрессия

- предсказываем по набору признаков нужное значение (или набор значений)

Кластеризация

- разбиваем объекты на группы схожих или “близких” без обучающей выборки

Комбинаторные задачи

- находим кратчайший путь, составляем расписание, строим график поставок - подбираем оптимальные параметры из большого множества вариантов

Регрессия

- предсказываем цену сделки
- предсказываем трафик машин / число покупателей на кассе
- генерируем изображения
- создаем музыку
- рекомендуем товары в интернет-магазины
- предсказываем популярность поста в соц сети
- предсказание цены квартиры
- предсказание объема продаж товара
- восстановление пропущенных значений
- предсказание стоимости акции
- предсказание параметров изделия
- предсказание выручки по клиенту
- предсказание координат объекта на изображении
- и др.

Классификация

- отличаем на фото кошек и собак
- отличаем транзакции фрод не фрод
- назначаем психотип клиентам
- предсказываем следующее слово в последовательности
- кредитный скоринг (вернет / не вернет кредит)
- склонность к хищению (украдет / не украдет)
- отток пользователей (уйдет не уйдет)
- сегментация клиентов (на какой типаж больше похож клиент)
- рекламное предложение (какой товар предложить следующим, какой тип акции применить)
- uiplift модели - давать или не давать скидку клиенту
- и т.д.

Нужна обучающая выборка (и чем больше тем лучше, обычно)

Кластеризация / сегментация

- ищем похожие объекты
- ищем нужные объекты
- ищем ответы на вопросы (поисковая система)
- ищем дубли
- строим рекомендации
- разбиваем на группы

Метрики для бинарной классификации

Задача: сбиваем самолеты, делая выстрелы. Должны сбивать вражеские, но если ошиблись, сбиваем свои…

Точность (precision)

- процент попаданий во вражеские по отношению к числу выстрелов (из 100 выстрелов, 70 попали во вражеские - точность 70%)

Полнота (recall)

- процент попаданий во вражеские по отношению к вылетам вражеских самолетов (из 200 вражеских самолетов мы сбили 70 - полнота 35%)

Компромисс

Чем больше точность - тем меньше полнота

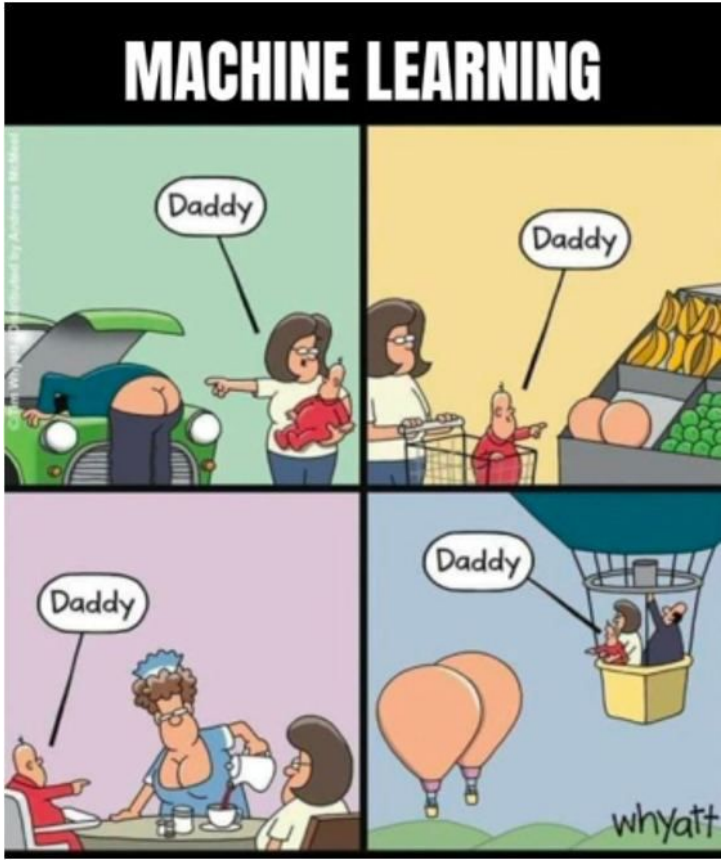
- стреляем когда уверены что вражеский только на 99% (те не стреляем когда уверены на 70%, 50% и т.д.)

Чем больше полнота - тем меньше точность

- сбиваем всех, кто хоть как-то может быть похож на вражеский (те стреляем, если уверены всего на 50%, 30%...)

Проблемы моделей

- обращают внимание на неверные или случайные признаки
- мы не знаем на что модель обратила внимание
- что показали, за то и зацепится



Проблемы моделей

- модель не “понимает” реальный мир.

- она цепляется за те данные, что были в обучающей выборке

- данные отличаются от обучающих - все ломается

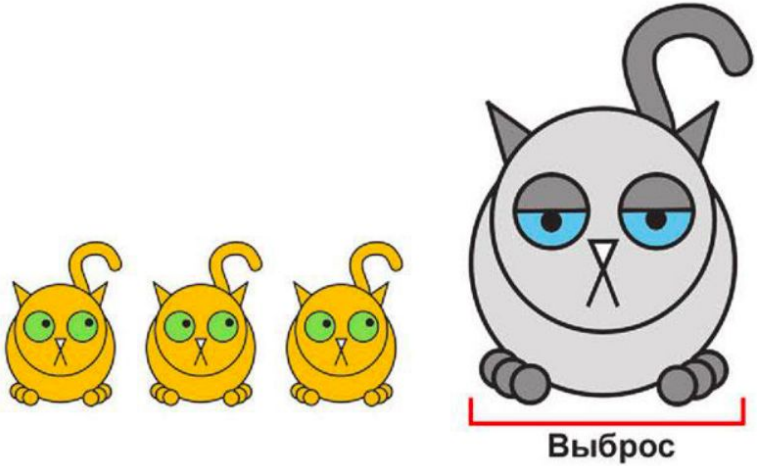
Сбор и подготовка данных - обычно не менее 50% работы

- сбор данных из различных источников
- построение feature-set (feature-engineering)
- построение data-set
- исследование данных (EDA)
- форматирование данных
- заполнение пропусков
- фильтрация данных
- дедупликация данных
- аугментация данных
- feature-selection
- построение процесса сбора данных (ETL-pipeline)

Чистка данных

- Обработка пропущенных значений (удаление, заполнение средними, медианой, модой и т.д.).
- Приведение данных к единому формату (даты, единицы измерения и т.д.).
- Преобразование категориальных признаков в числовые (one-hot encoding, label encoding и т.д.).
- Масштабирование и нормализация признаков.
- Приведение данных к единой базе, если вычисляются по разной методике.

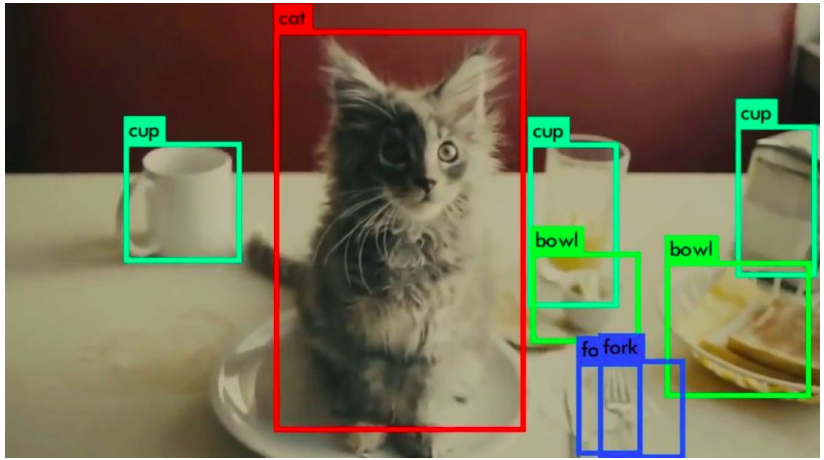
Выбросы - нетипичные значения во входных данных, которые могут привести к некорректной работе модели



Проблемы обучения

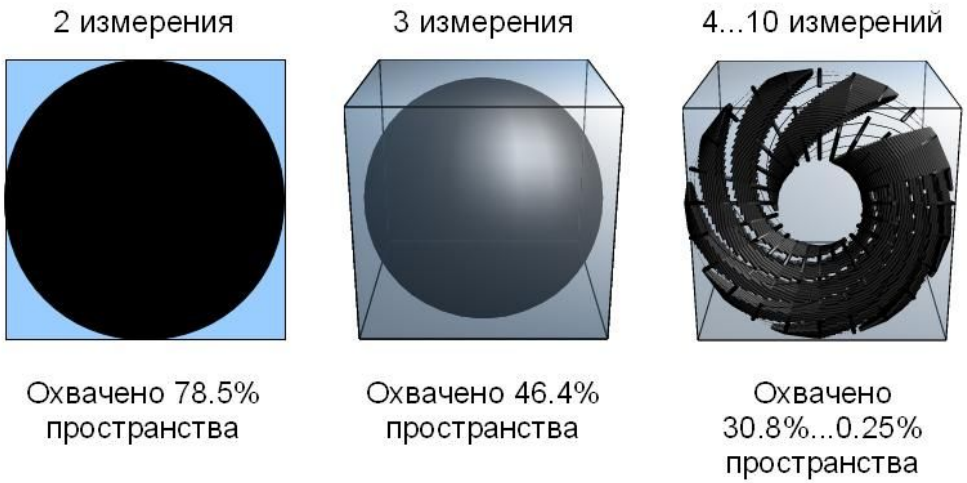
- несбалансированность классов
- under-sampling
- over-sampling
- SMOTE (генерация синтетических данных)

- шумные данные
- ложные срабатывания



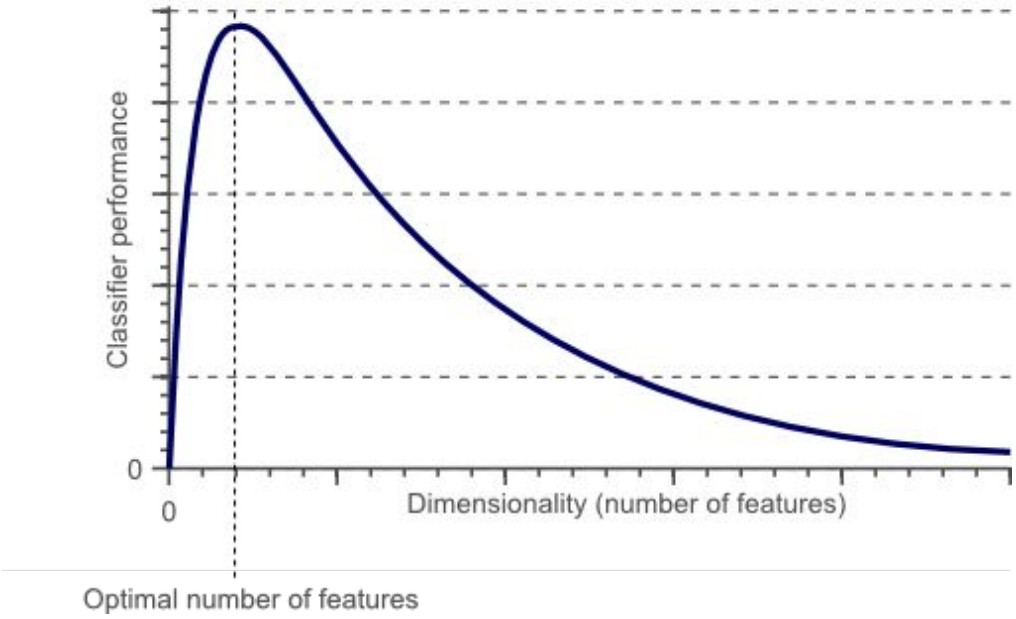
Проклятие размерности

- чем больше признаков (число признаков = размерность), тем сложнее отличать объекты
- когда признаков больше 3-х визуализировать практически невозможно
- когда больше 100 признаков, все от всего примерно равноудалено

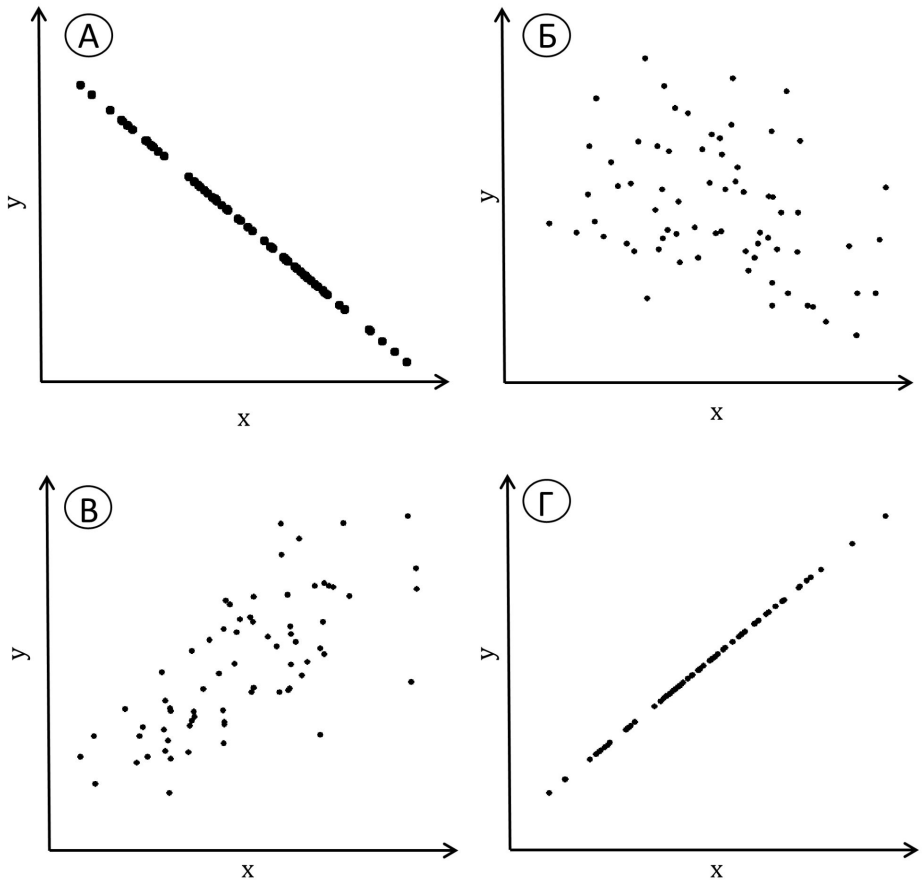


Фича инжиниринг

- какие признаки оставить?
- это большая работа - предобработка данных и отбор признаков
- правильно подобранные признаки снижают шум и повышают качество
- каждый раз это поиск, идеального рецепта нет

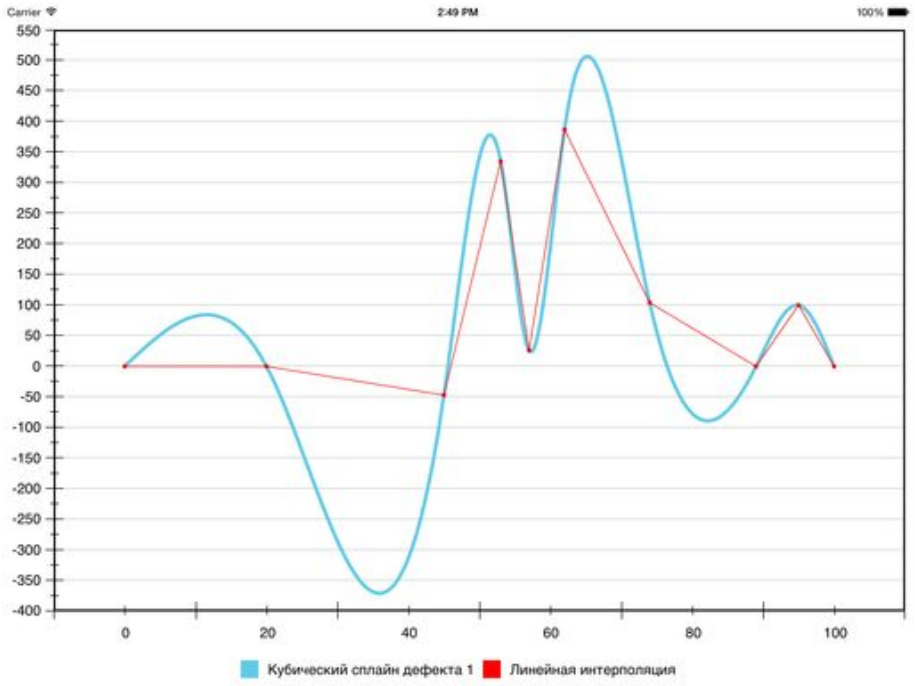


- насколько целевая закономерность ясна?
- ваша модель может вытащить только тот сигнал, что есть в ваших данных



Сложная форма связи

- чем сложнее связь тем более мощная по разрешающей способности модель нужна
- тем больше нужно обучающих данных
- тем тяжелее модель в инференсе



Что есть модель?

f(X) - некоторая функция (порой достаточно сложная).

- которая по набору x1, … , xN

  должна дать Y0

- но она дает Y которое не равно Y0

  ошибка

  e = Y - Y0

Гиперпараметры модели

Для модели
- класс модели
- размер модели
- количество входных параметров модели
- количество внутренних параметров (весов) модели
- форма нелинейности
- количество слоев (деревьев, слагаемых)
- ограничения на коэффициенты
- и т.д.

модель типа “решающее дерево”

Есть готовые библиотеки для обучения моделей

sklearn - формулы, деревья
scipy - статистические модели
CatBoost - ансамбли деревьев
XGBoost - ансамбли деревьев
Keras - нейронные сети
PyTorch - нейронные сети
…

```
from lib import model

model.fit()
model.predict()
```

Обучаем модель

```
from sklearn.model_selection import train_test_split

X = data.drop('Outcome', axis = 1)
y = data['Outcome'].astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42, st
ratify = y)
```

```
model = LogisticRegression(learning_rate= 0.1, no_of_iterations= 200)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
print(f'Accuracy: {accuracy(y_test.values, y_pred)}%')
```

```
Accuracy: 78.5%
```

Что внутри модели?

f(X)

внутри веса и вычислительный граф

f = 1*x1 + 2*x2 + … + 3*x50

x1, …, x50 - входные параметры
1, 2, …, 3 - веса

f(X) = Y0 - должно быть в идеале, например Y0 = 100
f(X) = Y - реально дает Y = 110

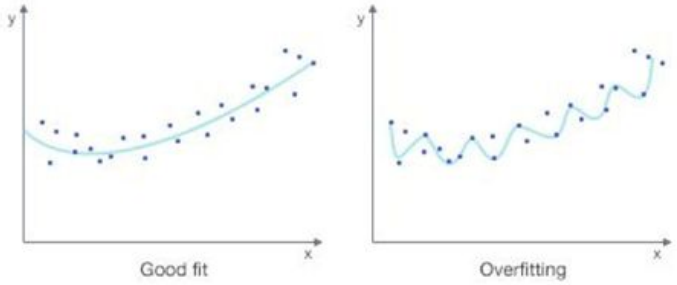
модуль ошибки |e| = 10

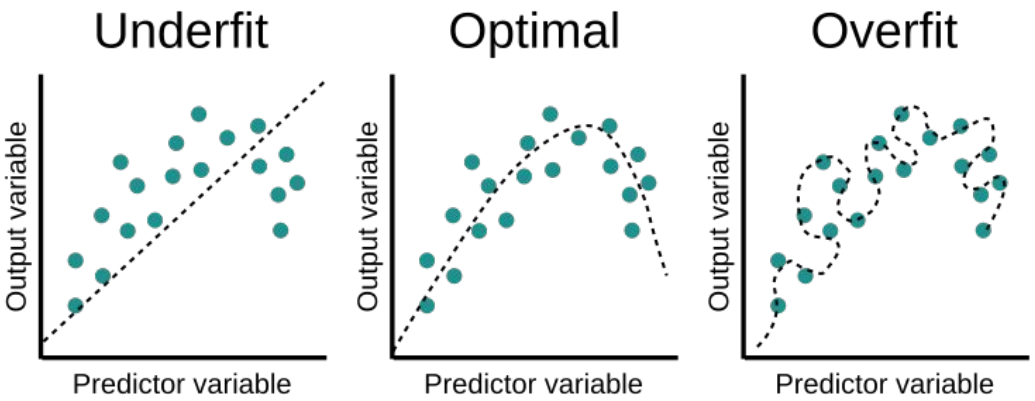
Обучение модели = подбор наилучших весов

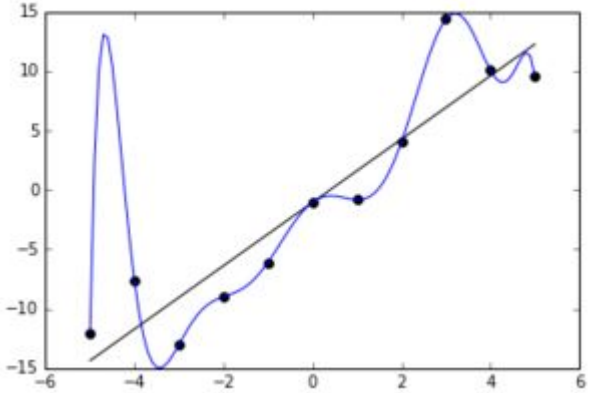
Переобучение

Для контроля разбиваем данные на трейн и тест.

Но и трейн, и тест могут не заметить переобучение



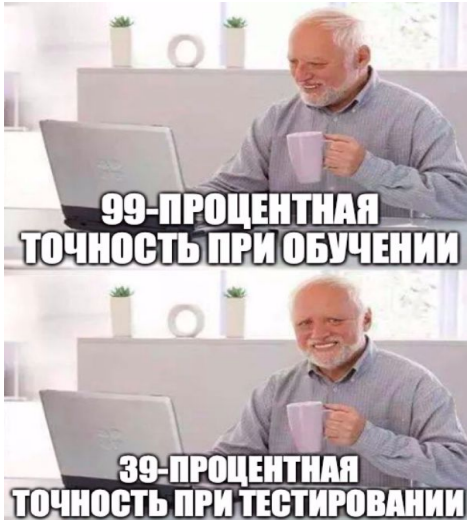




Проверка модели

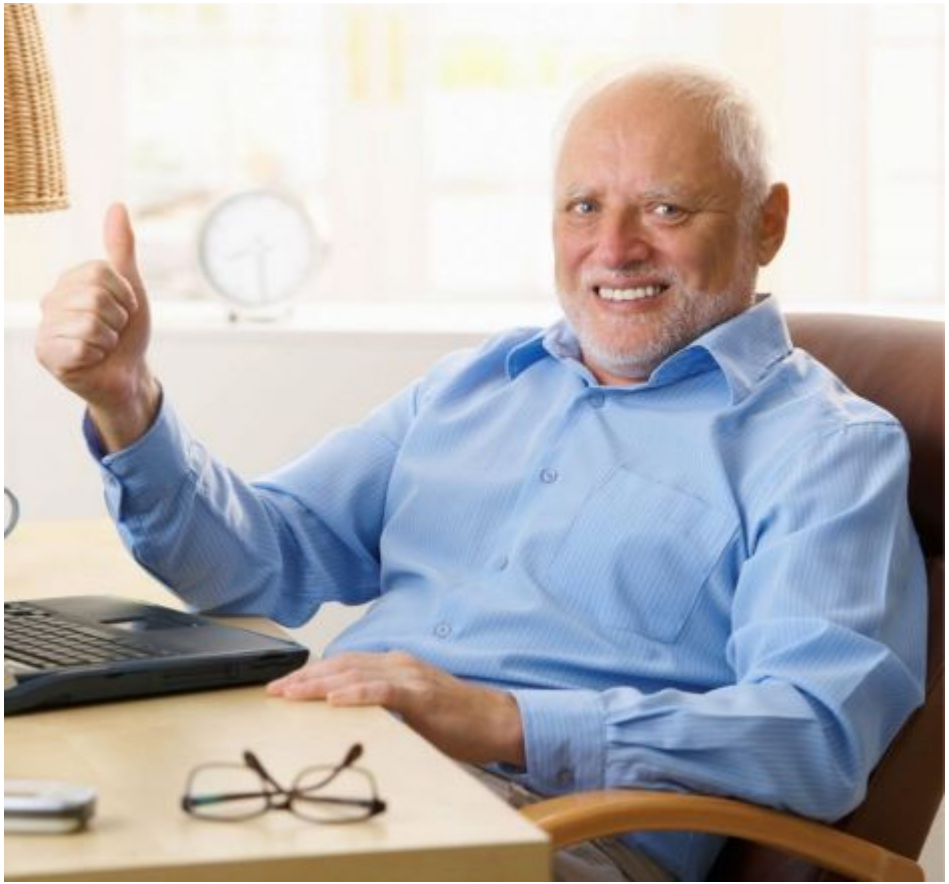
Тренировочная выборка - выборка, на которой мы подбираем веса

Тест выборка - на которой мы проверяем модель



Довольно часто мы не можем найти даже модель, которая на тесте отработает хорошо…

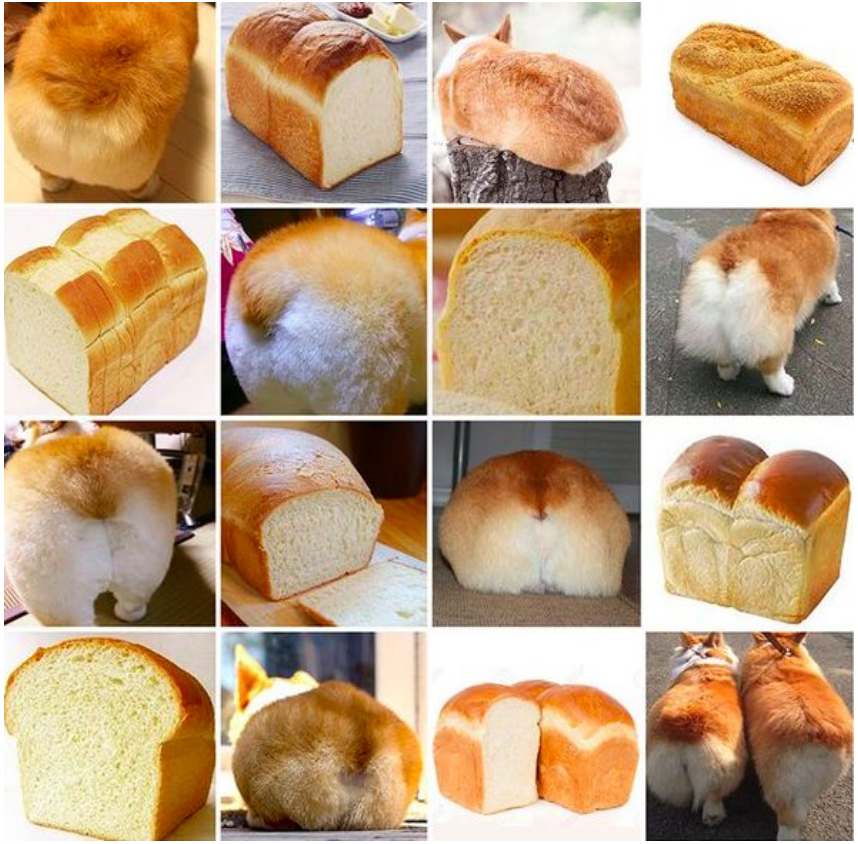
Если в данных мало закономерности, связи, сигнала…



Данные в проде всегда! отличаются от данных на обучении.

Данные в проде со временем меняются (дрифт данных)

Модели протухают

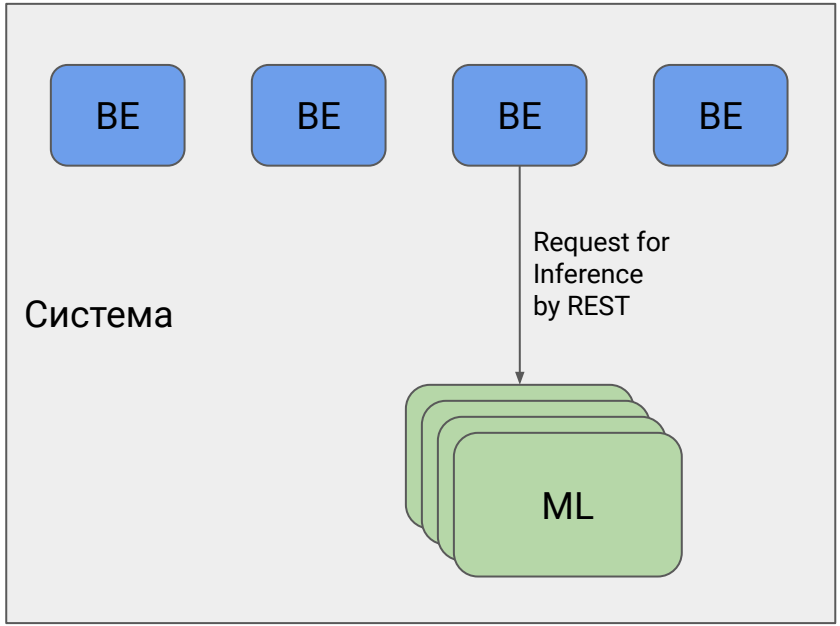


Интеграция ML в BE

Модель как микросервис

Идеальная схема интеграции

1) Отдельный микросервис (MS) с ML-моделью на Python - REST API Сервис
2) Используем Python: Flask, FastAPI
3) Модель живет в Docker контейнере
4) Бэкенд взаимодействует с ML-MS по HTTP для run-time
5) Или модель запускается в Docker по расписанию для пакетного исполнения



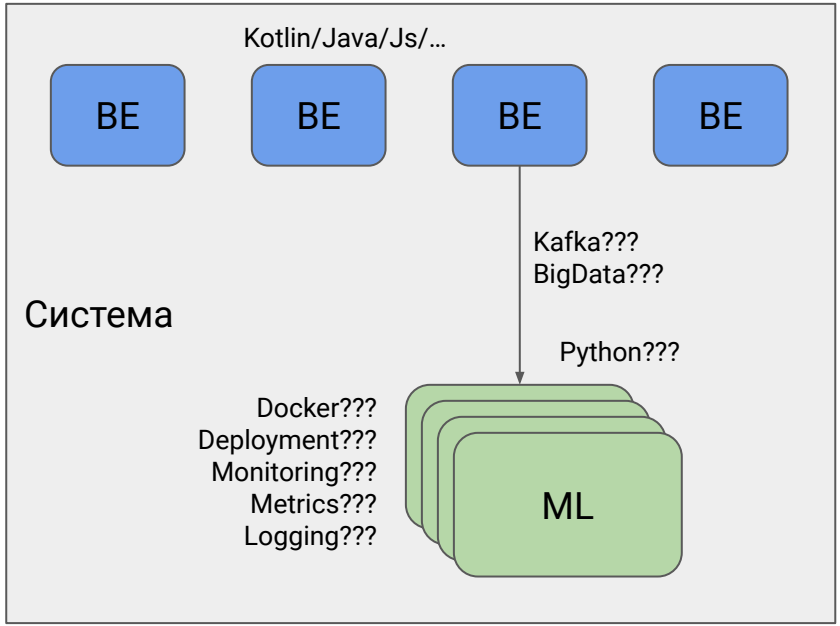
Основные проблемы интеграции ML-BE

Идеальная схема интеграции

1) Отдельный микросервис (MS) с ML-моделью
2) Бэкенд взаимодействует с ML-MS по HTTP

Реалии

1) Дата-инженеры не обязаны быть специалистами по разработке High-Load (кэш, rps)
2) Не всегда подходит интеграция по HTTP
3) Требуется много дополнительных обвязок: мониторинг, метрики, тестирование, контейнеры, логирование и пр.
4) Разный стек на BackEnd (Java/JS/…) и в ML-MS (Python)



Нативные библиотеки для ML (JVM-библиотеки)

Преимущества:

- Нет необходимости в межсервисном взаимодействии, что снижает задержку.
- Уменьшение сложности развертывания (один сервис).

Недостатки:

- Ограниченный набор доступных библиотек и инструментов по сравнению с Python-экосистемой.
- Увеличение размера бэкенд-приложения.

Deep Learning for Java

Deep Learning for Java

https://ru.wikipedia.org/wiki/Deeplearning4j

Deep Learning for Java (DL4J) — это мощная библиотека для глубокого обучения и машинного обучения, разработанная для JVM (Java Virtual Machine) и поддерживающая языки Java и Kotlin. DL4J является частью более широкой экосистемы, включающей библиотеки для численных операций, обработки данных и научных вычислений.

Deep Learning for Java

Эксперименты на Python - продакшен модель на целевом языке

Все эксперименты с моделями в Python

- понимаем максимально достижимые метрики
- понимаем архитектуру и гипер-параметры модели
- финальную модель для прода учим в целевой экосистеме, нативной для продакшена.

ONNX - open neural networks exchange

ONNX (Open Neural Network Exchange) позволяет преобразовывать модели из различных фреймворков (TensorFlow, PyTorch, Sklearn и др.) в единый формат.

Можно использовать ONNX Runtime для загрузки и выполнения модели в Kotlin-приложении.

Преимущества:

- Поддержка различных фреймворков.
- Эффективное выполнение модели.

Недостатки:

- Дополнительный шаг преобразования модели в ONNX формат.
- Ограниченная поддержка некоторых специфических операций и функций.

ONNX

https://onnxruntime.ai/docs/get-started/

ONNX - open neural networks exchange

https://onnx.ai/supported-tools.html

https://github.com/onnx/onnxmltools

ONNXMLTools enables you to convert models from different machin learning toolkits into ONNX. Currently the following toolkits are supported:

- Tensorflow (a wrapper of tf2onnx converter)
- scikit-learn (a wrapper of skl2onnx converter)
- Apple Core ML
- Spark ML (experimental)
- LightGBM
- libsvm
- XGBoost
- H2O
- CatBoost

Pytorch has its builtin ONNX exporter check here for details

ONNX - сериализация в ProtoBuf

Protocol Buffers — протокол сериализации (передачи) структурированных данных, предложенный Google как эффективная бинарная альтернатива текстовому формату XML.

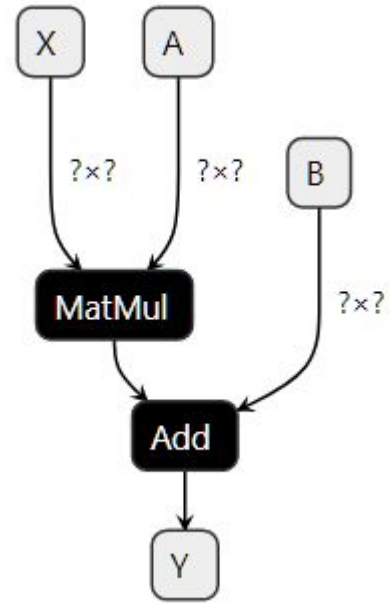
Портирован в C++, Java, Python, Go и другие языки.

ONNX описывает обученную модель в виде
- набора объектов входа
- набора объектов выхода
- графа вычислений

https://onnx.ai/onnx/operators/index.html

ai.onnx | ai.onnx.ml | ai.onnx.preview.training

operator	versions	differences
Abs	13, 6, 1	13/6, 13/1, 6/1
Acos	22, 7	22/7
Acosh	22, 9	22/9
Add	14, 13, 7, 6, 1	14/13, 14/7, 13/7, 14/6, 13/6,
AffineGrid	20	
And	7, 1	
ArgMax	13, 12, 1	
ArgMin	13, 12, 1	
Asin	22, 7	
Asinh	22, 9	
Atan	22, 7	
Atanh	22, 9	
AveragePool	22, 19, 1	
BatchNormalization	15, 14, 9,	
Bernoulli	22, 15	
BitShift	11	



ONNX Runtime

Оптимизированный инференс на CPU

Инференс на GPU

https://onnxruntime.ai/

Python | C# | JavaScript | Java | C++ | More..

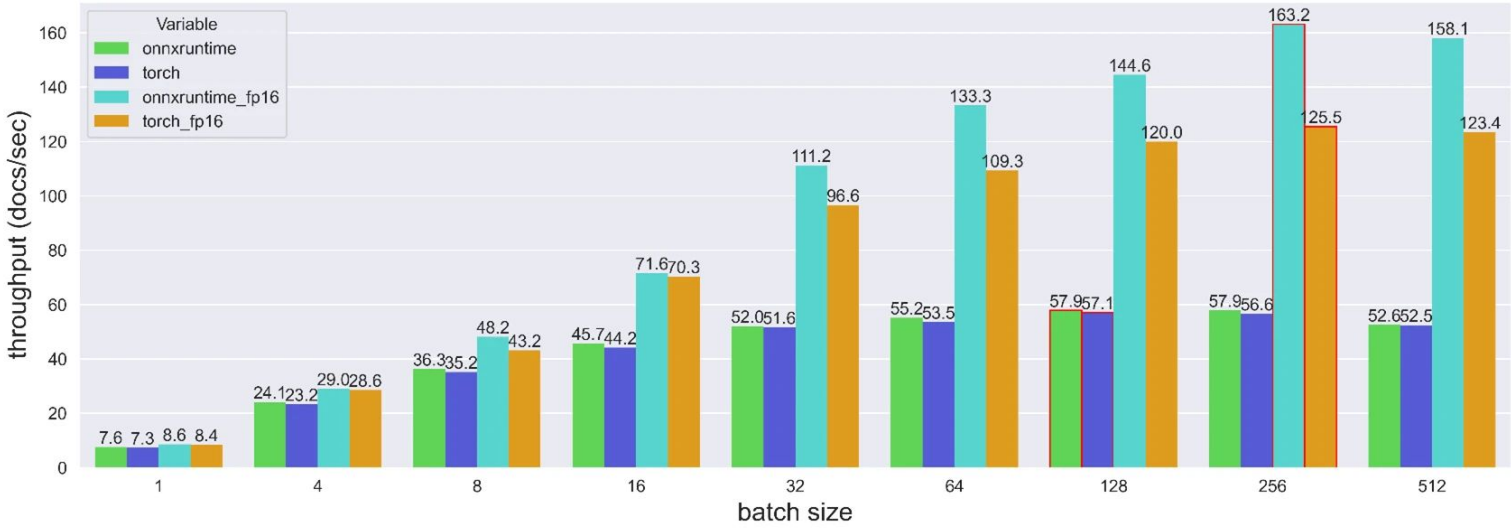
```
import ai.onnxruntime.*;
// Load the model and create InferenceSession
String modelPath = "path/to/your/onnx/model";
OrtEnvironment env = OrtEnvironment.getEnvironment();
OrtSession session = env.createSession(modelPath);
// Load and preprocess the input image to inputTensor
...
// Run inference
OrtSession.Result outputs = session.run(inputTensor);
System.out.println(outputs.get(0).getTensor().getFloatBuffer().get(0));
```

- Python
- C++
- C
- C#
- Java
- JavaScript
- Objective-C
- Julia and Ruby APIs
- Windows
- Mobile
- On-Device Training
- Large Model Training

ONNX Runtime - Квантование

Понижение точности вычислений модели на инференсе

- модель занимает меньше памяти
- вычисляется быстрее
- float64 -> float32 -> float16 / bfloat16 | (а ещё бывает -> int8 -> int4 -> int3)

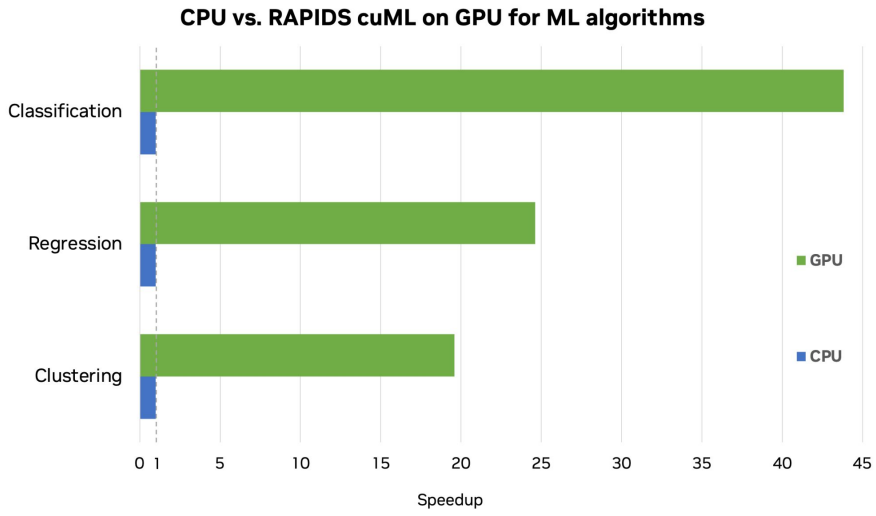


GPU

Параллельная обработка

Обучение нейронных сетей включает многочисленные матричные операции (умножение матриц, свертки и т.д.).

GPU могут одновременно выполнять многие такие операции, что значительно ускоряет процесс обучения.



GPU

1. cuDNN (CUDA Deep Neural Network library)

- Nvidia предоставила библиотеку cuDNN, оптимизированную для глубокого обучения, которая может быть интегрирована с различными фреймворками, такими как TensorFlow и PyTorch.

2. NVIDIA TensorRT

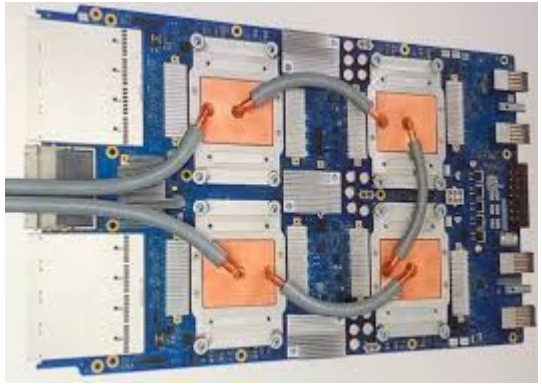
- Оптимизатор и ускоритель инференса нейросетей от Nvidia.
- Применяется для максимизации производительности моделей на GPU при инференсе.

3. ROCm (Radeon Open Compute)

- AMD предлагает платформу ROCm для вычислений на своих GPU.
- Поддерживает популярные библиотеки и фреймворки для глубокого обучения, такие как TensorFlow и PyTorch.

4. TPU для TensorFlow

- TensorFlow напрямую поддерживает Tensor Processing Units от Google, которые могут значительно ускорить обучение и инференс крупных моделей.

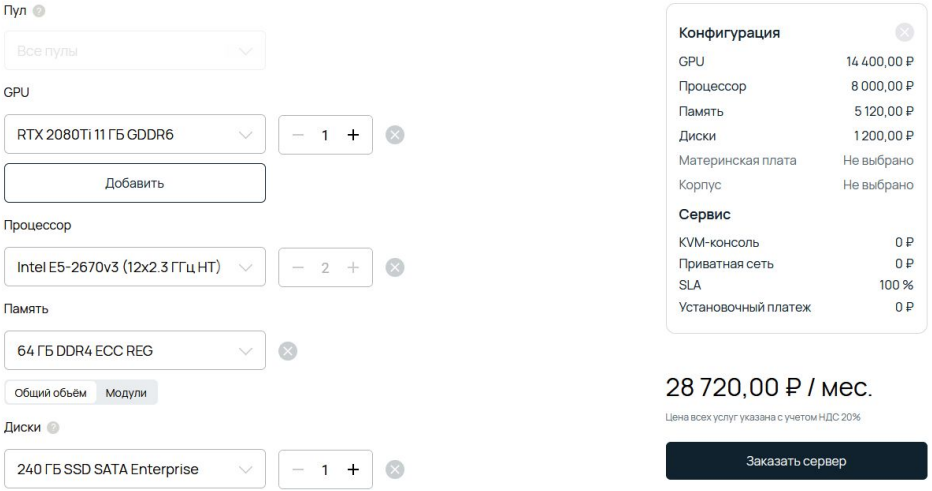


Тензорный процессор Google

Inference Server

Облачная или виртуальная машина на Linux

- с GPU



Selectel

Облака для обучения с мощным GPU

Compute Cloud | Object Storage | Kubernetes® | PostgreSQL | ClickHouse | MongoDB | MySQL®

Redis™ | Elasticsearch | Apache Kafka® | Greenplum® | GitLab® | SpeechKit | Translate | Tracker

SmartCaptcha | Техническая поддержка

Compute Cloud

Платформа	AMD EPYC™ with NVIDIA® Ampere® A100
Количество GPU	1 2 4 8
Количество vCPU	28
Объём RAM	119 ГБ
Прерываемая ВМ	☐
Публичный IP-адрес	☐
Исходящий трафик	0 ГБ
Диск – 1 (Загрузочный)	HDD SSD
Размер диска	1085 ГБ (5 ГБ – 4096 ГБ)

Добавить диск

Итого:

Compute Cloud × 1	322 680,74 ₽
AMD Epyc with Nvidia A100. 100% vCPU	18 144,00 ₽
AMD Epyc with Nvidia A100. GPU	267 624,00 ₽
AMD Epyc with Nvidia A100. RAM	23 990,40 ₽
Быстрое сетевое хранилище (SSD)	12 922,34 ₽
Исходящий трафик в интернет	0,00 ₽
322 680,74 ₽	в месяц

Перейти в консоль

Поделиться расчётом

Получить консультацию

Жизненный цикл модели

- собираете данные
- определяете метрики качества модели
- перебираете архитектуры / обучаете модель
- сервис инференса модели
- логирование инференса (полное / частичное)
- сбор тестовых данных на основе продакшена с ручной разметкой
- сравнение логов инференса с тестовыми данными
  - отбор краевых случаев
  - анализ качества модели
  - контроль дрифта данных (сравнение распределений данных во времени)
- обучение на новых данных
- снова сервис инференса модели
- … и далее по циклу

Жизненный цикл модели

- регулярный (еженедельный / ежемесячный) ETL pipeline сбора датасета
  - трейн части
  - тест части
  - контроль стаблильности распределения данных трейн / тест / прод
- обучение модели на новых данных
- сравнение метрик предыдущей и новой модели на тест части
- логирование метрик
- инференс модели с лучшими метриками на тесте
- логирование инференса
  - анализ распределений признаков
  - выборочный контроль

Основные ML модели

Классический ML

- “Небольшие” модели
- Неглубокие модели
- Вы строите (учите) модели сами на своих данных

- Нужен датасет
- Нужны сбор и разметка данных
- Нужно определить метрики
- Нужен процесс обучения модели - не выйдет найти готовую модель

Классический ML

- У нас всегда есть датасет - таблица, где есть N (10, 100, 1000, 10000) признаков (фичей) для каждого объекта и несколько целевых значенией (чаще один) - таргет.
- Задача научиться вычислять таргет по фичам.

```
frame
```

	fLength	fWidth	fSize	fConc	fConcl	fAsym	fM3Long	fM3Trans	fAlpha	fDist	Class
0	29.4491	12.7271	2.6637	0.3536	0.1876	-19.4070	-18.1295	7.1258	8.1501	252.1500	g
1	51.5830	10.7969	2.6222	0.5227	0.2733	-64.0583	-30.1280	4.3855	15.0428	269.4810	g
2	36.3558	10.3843	2.8531	0.5309	0.3599	15.4179	51.9328	16.2848	9.1263	208.7935	h
3	37.2577	12.0793	2.4354	0.3560	0.2037	5.1882	-17.8545	6.0370	30.0150	61.1727	h
4	34.8906	15.7072	2.7147	0.3587	0.1938	-8.5682	-12.6514	-14.3676	89.7920	222.4690	h
5	60.4957	17.6753	2.9380	0.1753	0.0894	31.3257	-15.9801	14.4117	15.6390	113.1820	g
6	18.5731	16.2365	2.6758	0.4895	0.3091	1.1158	8.1104	-11.7988	50.9791	224.8780	h
7	21.5929	12.4539	2.3512	0.4900	0.3096	0.1172	-4.3583	2.5525	58.3290	52.0772	g
8	45.5590	9.9957	2.5809	0.3885	0.1955	17.0284	34.9608	-7.8856	14.4173	177.6820	g
9	62.3597	21.6946	3.1739	0.3087	0.2041	-45.3412	52.1023	22.5905	36.9876	274.7409	h

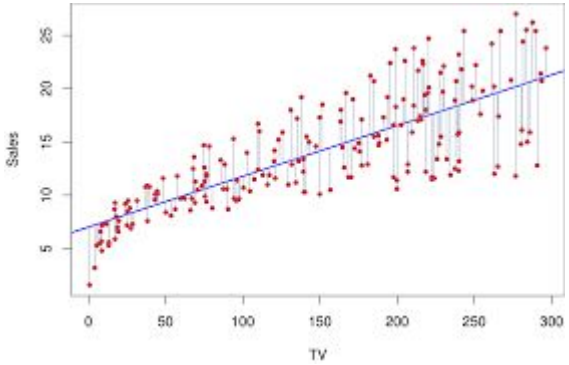
KNN - k nearest neighbors

- Скажи мне кто, твой друг и я скажу, кто ты.

- Запоминаем всю обучающую выборку.
- Для поступивших объектов ищем ближайших k (1,3,5,10,15) примеров
  - для регрессии целевое значение определяем как среднее по k соседям
  - для классификации вероятности классов оцениваем как распределение классов по соседям
- Близость вычисляем как обычное расстояние, например.
- Все признаки надо нормировать - например в диапазон [0,1] или в N(0,1)
- Хорошо когда мало обучающих примеров или верна гипотеза о локальности

Linear regression

- Попробуем таргет вычислять как линейное выражение от фичей
- y = a1*x1 + a2*x2 + … + aN*xN + b
- можем брать фичами не только x1, …, XN, но и различные их нелинейные комбинации: x1*x1, x1*x2, …, x1 / x2, …
- легко обучается, легко применяется
- но не устойчиво к выбросам
- надо масштабировать или нормировать данные
- хорошо, когда связь простая

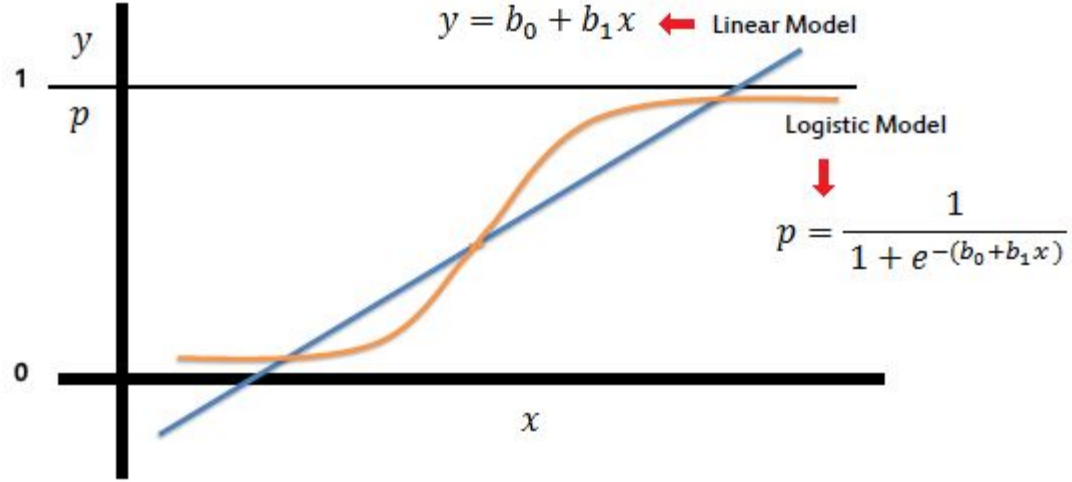


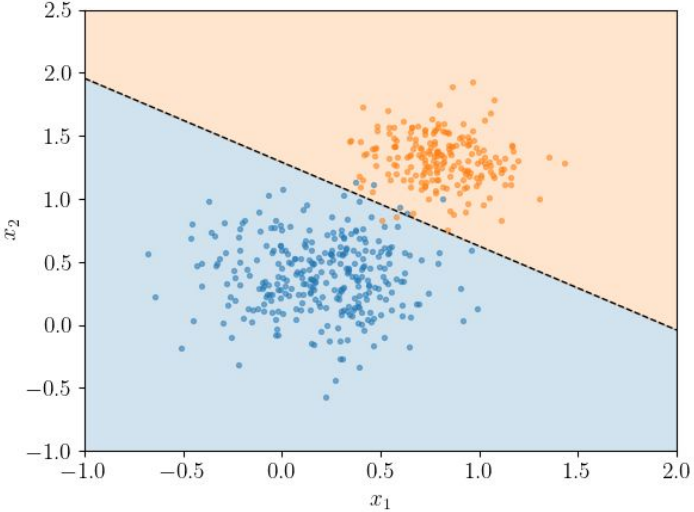
Logistic regression

- Как линейная регрессия, только для классификации на два класса - предсказываем метки 0 и 1
- Попробуем таргет вычислять как сигмоиду от линейного выражения фичей
- y = sigmoid(a1*x1 + a2*x2 + … + aN*xN + b)
- можем брать фичами не только x1, …, XN, но и различные их нелинейные комбинации: x1*x1, x1*x2, …, x1 / x2, …
- легко обучается, легко применяется
- но не устойчиво к выбросам
- надо масштабировать или нормировать данные

Logistic regression

- Предполагает, что классы линейно разделимы в пространстве фичей
- те можно провести гиперплоскость, что с одной стороны в основном один класс, а с другой стороны другой класс.





Категориальные и порядковые данные

- Мужчины, женщины
- Город
- Профессия
- Грейд
- …

- Label Encoding - нумеруем и заменяем числом
- One-Hot Encoding - каждое значение заменяем отдельной фичой (100 значений - плюс 100 фичей)

One-Hot encoding

- все надо превратить в числа
- кроме Label и One-Hot есть еще варианты кодирования

Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

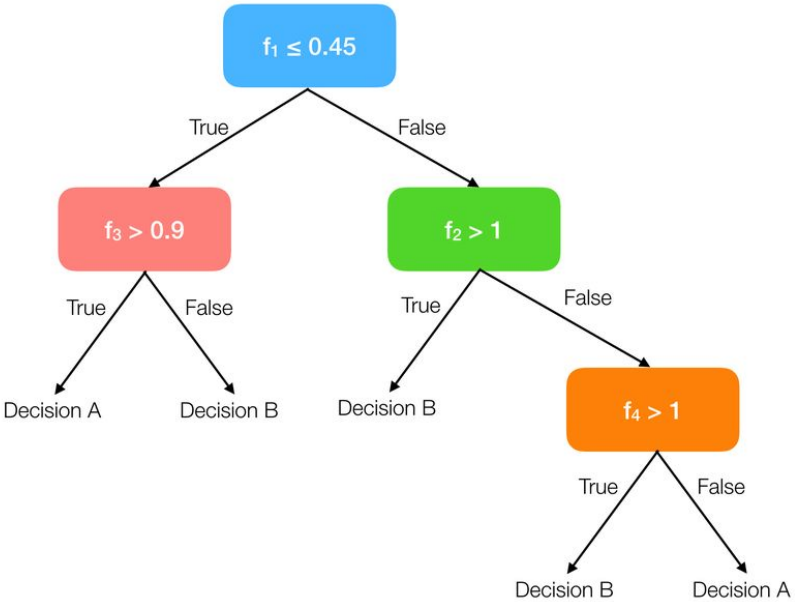
$\rightarrow$

One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

Decision tree

- Решающие деревья
- кусочно линейные функции
- можно разбить пространство на однородные куски, внутри которых “все понятно”
- можно ограничивать глубину дерева
- количество примеров в листе
- отлично интерпретируется
- легко программируется
- не боятся категориальных данных
- есть готовые библиотеки обучения

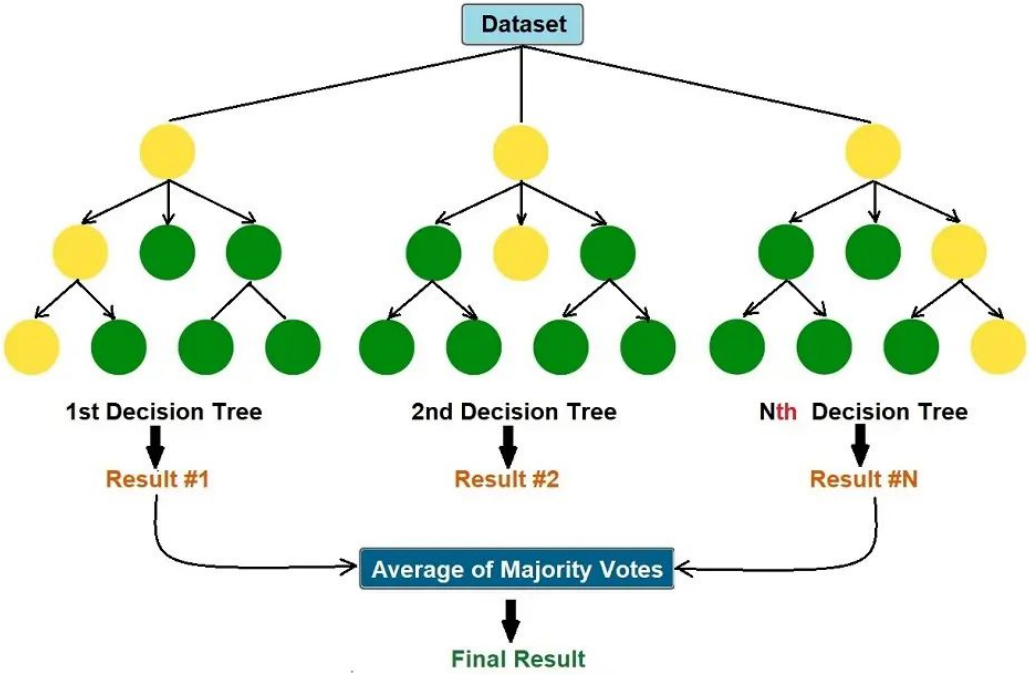


Random Forest

- Мудрость толпы
- Построим не одно дерево, а 10, 100 или 1000
- Чтобы деревья были разными - будем им на каждом шаге разделения показывать не все обучающие данные и не все фичи
- чтобы у каждого дерева было “свое мнение”
- взгляд на проблему без части данных и примеров, чтобы выделить наиболее общую закономерность
- итоговое решение - это усреднение предсказаний всех построенных деревьев на предложенном объекте

Random Forest

- не предскажем то, чего не видели
- ограничения MIN, MAX примеров на обучающей выборке
- не генерализуются на левый и правый хвосты
- деревья лучше делать не очень глубокими (3,5, 10, 15, 50 глубина)

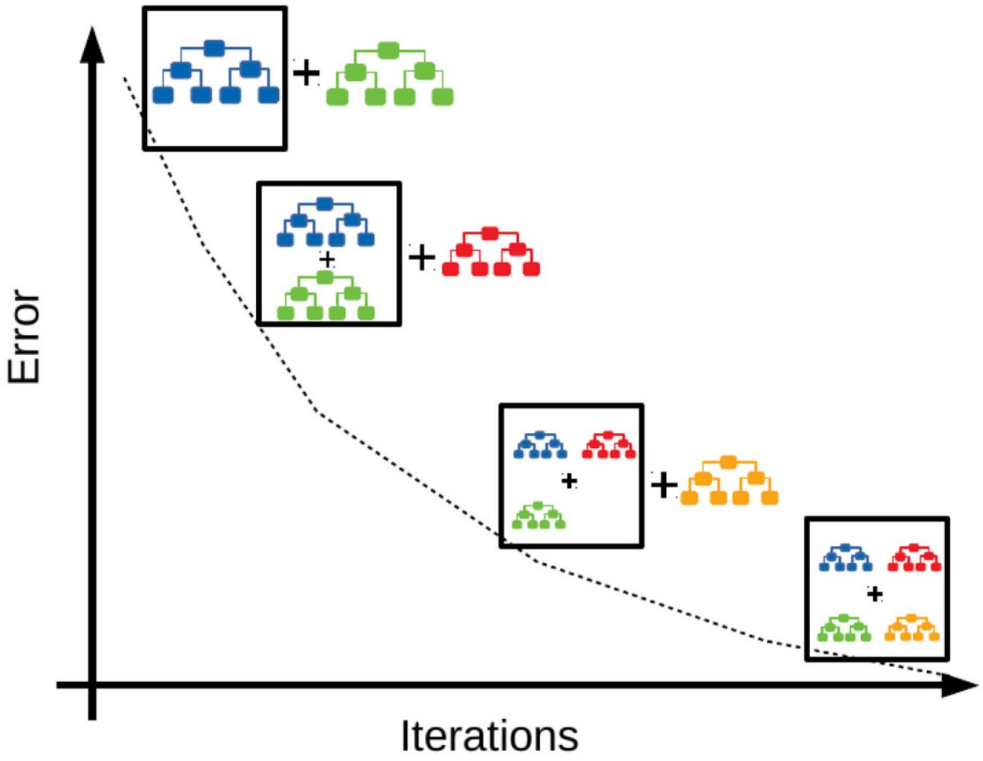


Random Forest

- модель устойчива к выбросам
- устойчива к переобучению
- не требует нормировки или масштабирования фичей
- умеет работать с категориальными и порядковыми переменными
- выносится в ONNX, реализована во всех ML библиотеках
- но может много весить
- и не очень быстро отрабатывать, особенно, если много фичей и деревьев

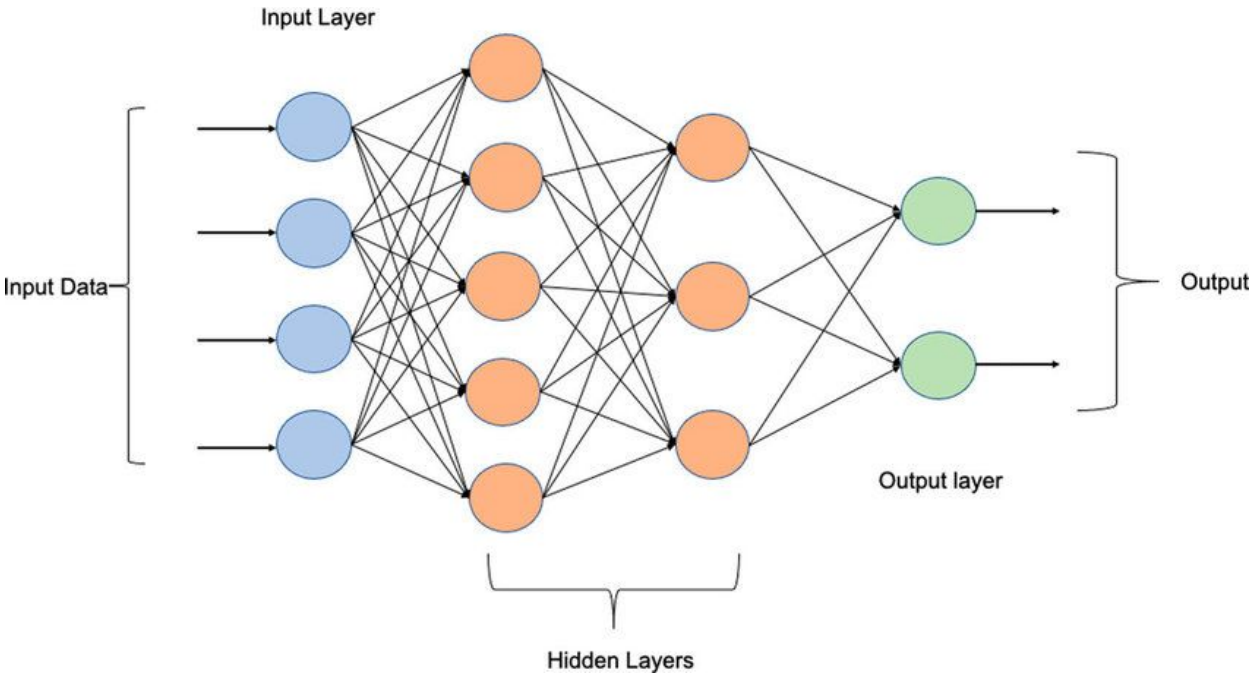
Gradient Boosting Tree

- Бустинговые деревья:
- построим дерево D1, предсказывающее таргет Y
- получим ошибку E1
- построим дерево D2, предсказывающее E1, тогда D1 + D2 предскажет таргет Y уже лучше
- получим ошибку E2
- построим дерево D3 на ошибку E2 и т.д.
- …
- Ypred = D1 + D2 + D3 + … + D100

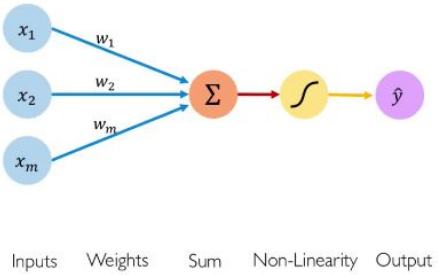


Неглубокие нейросети

- MLP - multi-layer perceptron



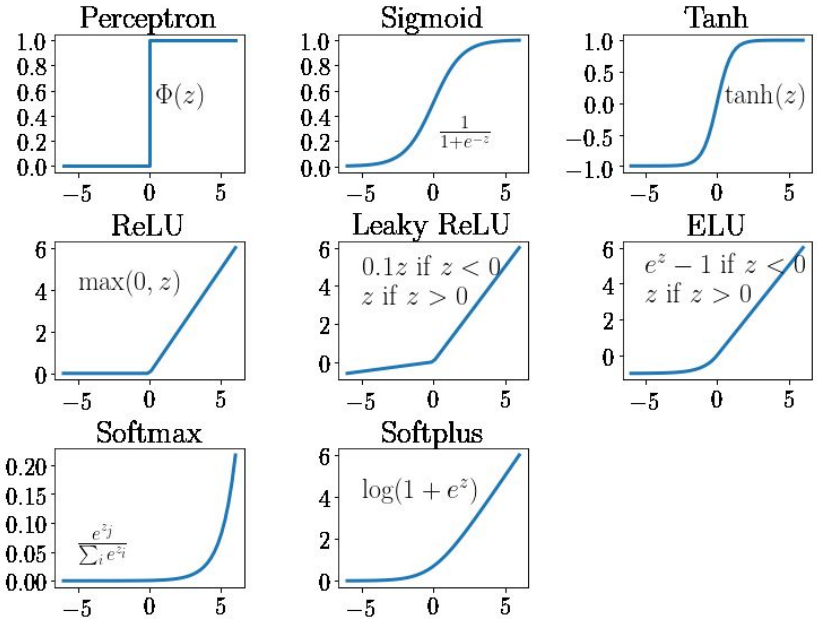
Один нейрон



Неглубокие нейросети

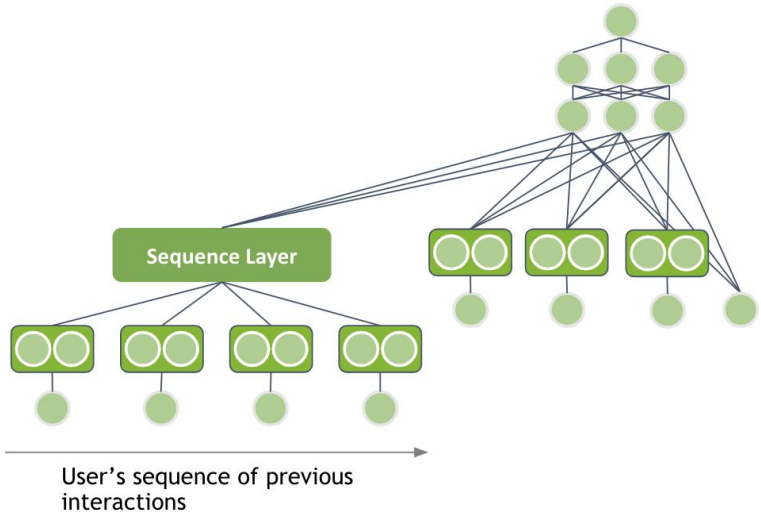
- Простые функции активации
- 1-5 слоев
- нужно не много данных для обучения (десятки тысяч примеров)
- быстро учатся
- быстро инферятся
- любят нормированные числовые данные
- регрессия, классификация, автоенкодер

Функции активации



Неглубокие нейросети

- Позволяют смешивать доменные данные
- можно сложить попу с пальцем:
  - смешать данные пользователя
  - данные о тексте товара
  - данные о истории просмотра
  - …
  - все сложить в одну модель и предсказать таргет, глядя одновременно на все это

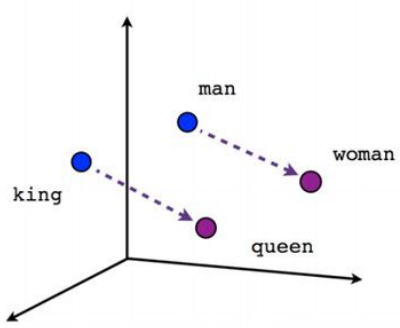


Работа с текстами - NLP

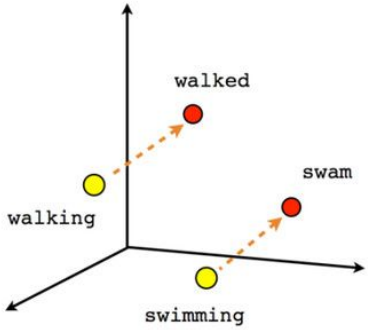
- Построение вектора текста, отражающего смысл текста
- Классификация текста
- Выделение из текста нужных сущностей
  - имена, организации, локации, действующие лица и проч
- Поиск по текстам
- Генерация текста (чат боты и др.)

Работа с текстами - Fast text

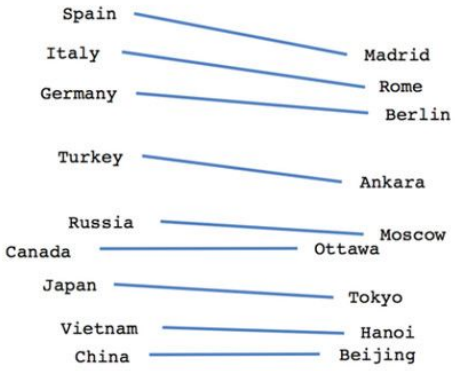
- Строим векторы слов. Близкие по тематике слова - имеют близкие векторы.
- Строим векторы коротких текстов, усредняя векторы слов текста
- Легко классифицировать тексты до 100 слов
- Есть предобученные модели
- Есть готовые библиотеки



Male-Female



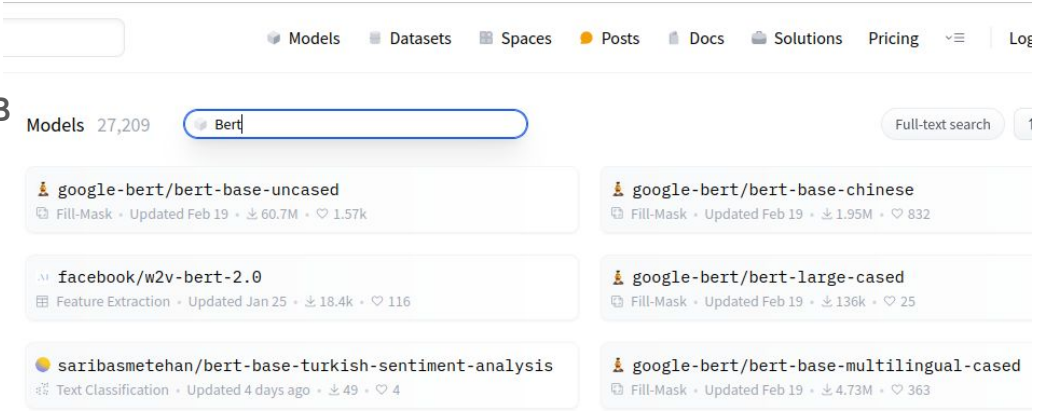
Verb tense



Country-Capital

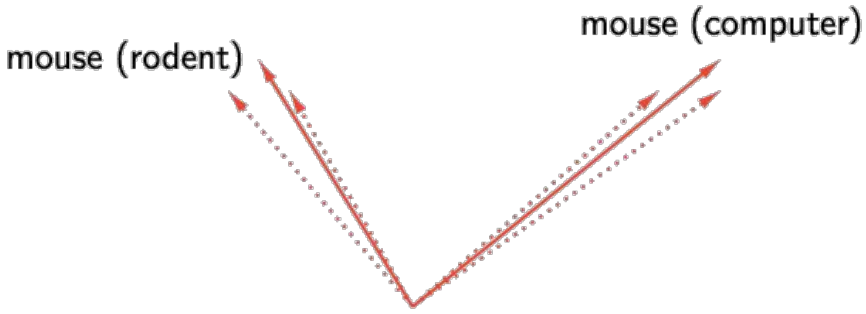
Работа с текстами - BERT (Трансформер)

- Строим векторы слов, учитывающие смысл целой фразы
- Предобученные модели для русского, английского и др. языков
- Можно использовать для средних текстов длиной в 100-500 слов
- https://huggingface.co/models
- Быстро работают на CPU
- Понимают смысл текста в целом
- Используются для поиска по текстам
- Классификации текстов
- Извлечения информации из текстов
- Матчинга и ранжирования текстов
- Выносятся в ONNX



Контекстные эмбеддинги

- смысл слова зависит от контекста: теннисный стол, кухонный стол
- улучшение качества машинного перевода и обработки естественного языка
- повышение точности задач классификации и извлечения информации
- обучение моделей с меньшим количеством данных

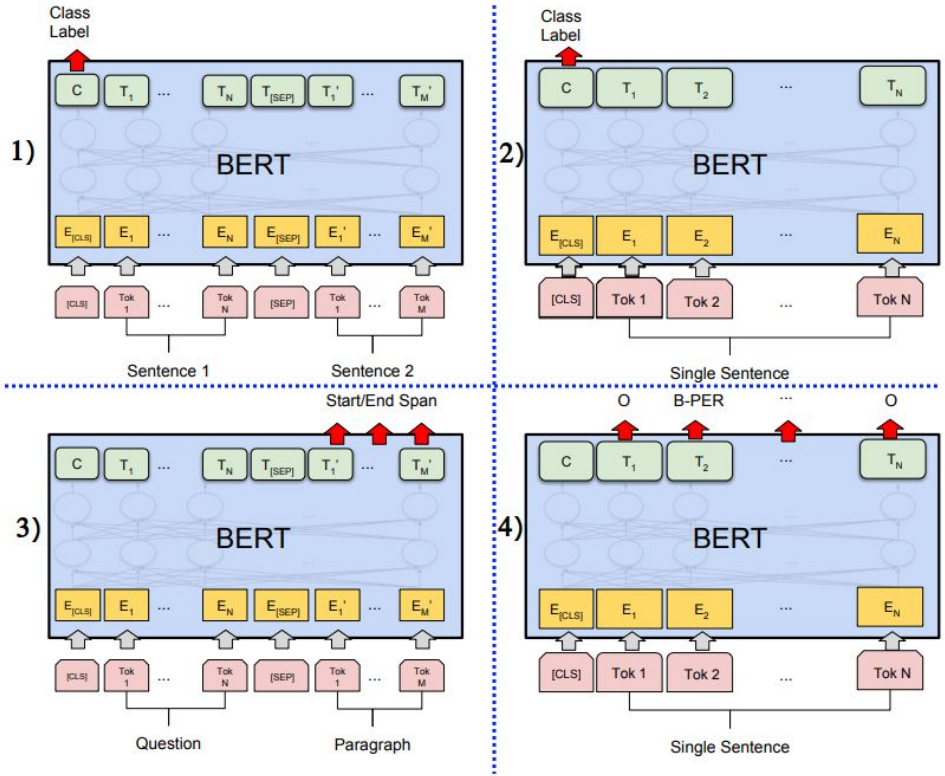


Контекстуализированные эмбеддинги слов позволяют моделям лучше понимать значения слов в контексте предложения и общий смысл предложения

BERT finetune

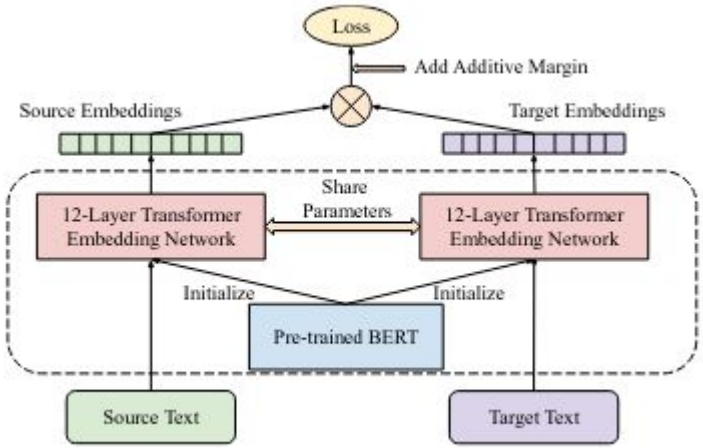
- Классификация
- NER
- QA
- Extraction
- Extractive summarization
- …
- Есть готовый эмбеддинг текста

от 50Мб до 2.5Гб



LABSE - готовые векторы текстов

- Близкие по смыслу тексты имеют близкие векторы
- Language Agnostic Bert Sentence Embeddings
- Близкие векторы для текстов на разных языках, если они значат одно и то же
- Можно научить модель чтобы ответ на вопрос имел близкий вектор к тексту вопроса
- Поиск по векторным базам данных

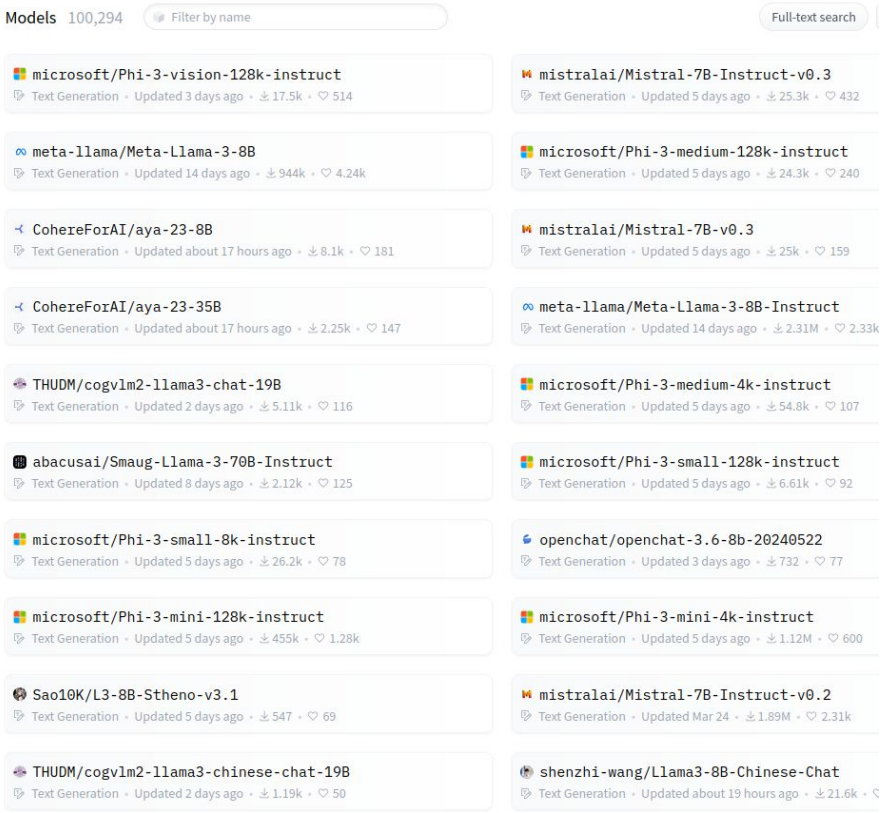


https://huggingface.co/cointegrated/LaBSE-en-ru

500Mb

LLM - большие языковые модели

- ИИ чат боты
- https://huggingface.co/models?pipeline_tag=text-generation&sort=trending
- open source
- API
- Chat GPT (GPT-4)
- Гигачат
- YandexGPT
- …

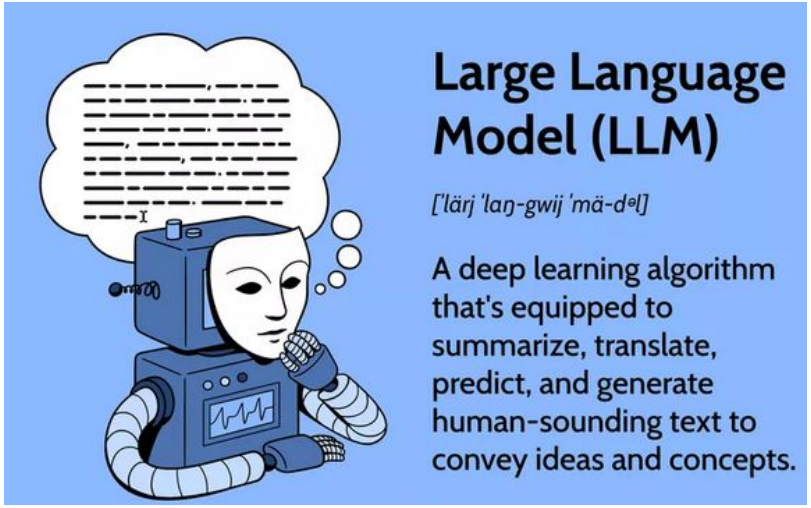


LLM - большие языковые модели

- LLAMA
- Mistral
- RuGPT3.5
- Phi

- Базовые
- Инструктивные

- LORA модификаторы (Сайга, Альпака, …)



LLM - большие языковые модели

- Промптинг
- Хорошо классифицируют
- Суммаризируют
- Выделяют сущности
- Отвечают на вопросы на основе контексты
- https://www.promptingguide.ai/ru

- Разбивать сложное на небольшие “атомарные операции”.
- Составлять промпты с примерами, которые помогут модели понять что вам нужно
- Работать через API

Перевод звука в текст - Whisper

- https://github.com/openai/whisper
- speech-to-text

Size	Parameters	English-only model	Multilingual model	Required VRAM	Relative speed
tiny	39 M	`tiny.en`	`tiny`	~1 GB	~32x
base	74 M	`base.en`	`base`	~1 GB	~16x
small	244 M	`small.en`	`small`	~2 GB	~6x
medium	769 M	`medium.en`	`medium`	~5 GB	~2x
large	1550 M	N/A	`large`	~10 GB	1x

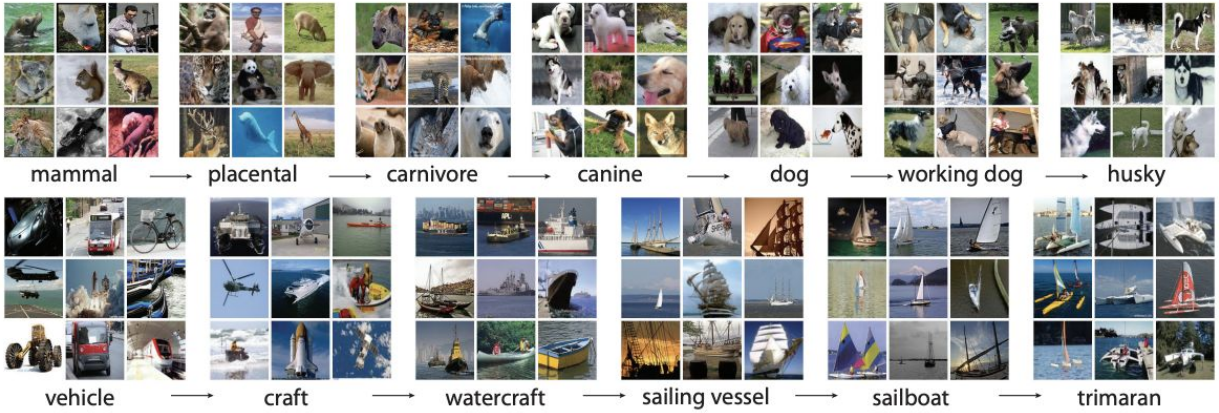
Работа с изображениями и видео - CV (Computer Vision)

- Детекция объектов на изображениях
- Классификация изображений
- Сегментация изображений
- Генерация изображений
- Захват и отслеживание объектов на видео

- Resnet
- Yolo
- StableDiffusion

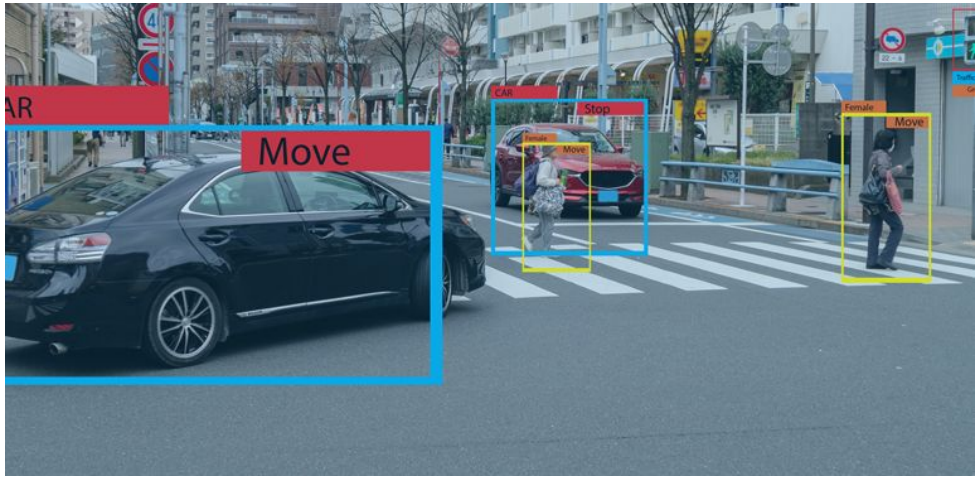
ResNet

- Классификация объектов на изображениях
- Получение вектора изображения
- Поиск похожих изображений
- Поиск дублей изображений
- Обучена на десяток тысяч классов изображений на ImageNet
- https://pytorch.org/hub/nvidia_deeplearningexamples_resnet50/
-



YOLO

- You Only Look Once
- https://github.com/ultralytics/ultralytics
- https://yolov8.com/
- YOLO 1, 2, …, 8, …
- Детекция объектов на изображении

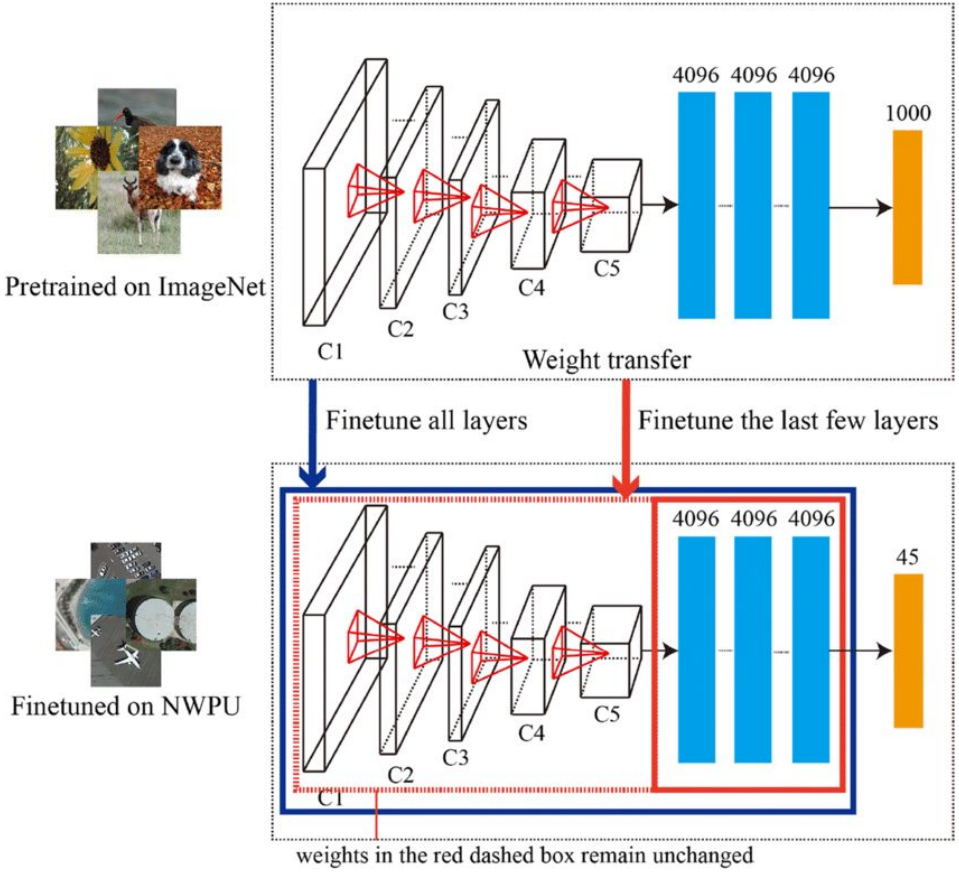


Stable Diffusion

- Генерация изображений по текстовому описанию
- https://github.com/Stability-AI/stablediffusion
- Можно генерировать на домашнем компьютере

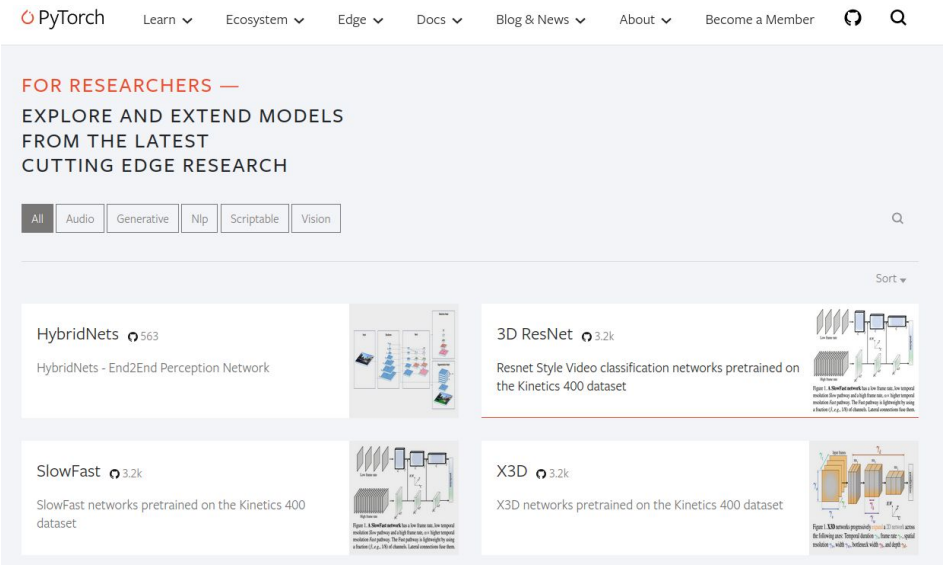
Fine-Tuning

- Берем большую умную предобученную модель
- Собираем свой небольшой датасет на 100-1000-10000 примеров
- Доучиваем модель на свою задачу
- Добавляем/заменяем последние слои
- Заморживаем часть большой модели, если данных мало



Где брать предобученные модели?

- GitHub
- Torch-hub
  https://pytorch.org/hub/
- Torchvision-hub
  https://pytorch.org/vision/0.8/models.html
- HuggingFace
  https://huggingface.co/models
- Tensor-Flow Hub
- …



Библиотеки в экосистеме Python

- Jupyter Notebook - среда разработки
- FastApi / Flask - для API
- Sklearn - для классического ML
- XGBoost / Catboost - бустинговые деревья
- Pytorch - нейросети
- Transformers - предобученные модели на HuggingFace
- virtualenv / Docker - для продуктивизации
- ONNX - для инференса и экспорта моделей

Спасибо за внимание

- Видео по машинному обучению
- Ссылки на бесплатные материалы и курсы по ML / Ds
- Карта компетенций Data Scientist

https://mlnavigator.ru/